RUDN university

Reinforcement Learning with Artificial neural network

Lecture 2

Prepared by

Lect. Safa Abdulwahab Hameed

Mechanics and process control engineering faculty

Outline

- **Reinforcement learning Introduction**
- **History of Reinforcement learning**
- **Reinforcement learning**
- **Reinforcement learning vs Supervised learning**
- **Reinforcement learning vs Unsupervised learning**
- **Examples**

Reinforcement learning Introduction

The idea that we learn by interacting with our environment is probably the first to occur to us when we think about the nature of learning.

When an infant plays, waves its arms, or looks about, it has no explicit teacher, but it does have a direct sensorimotor connection to its environment.

Exercising this connection produces a wealth of information about cause and effect, about the consequences of actions, and about what to do in order to achieve goals.

Throughout our lives, such interactions are undoubtedly a major source of knowledge about our environment and ourselves.

Whether we are learning to drive a car or to hold a conversation, we are acutely aware of how our environment responds to what we do, and we seek to influence what happens through our behavior.

Learning from interaction is a foundational idea underlying nearly all theories of learning and intelligence.

The nature of human intelligence has attracted the special interest of scientists and researchers, as humans are the most intelligent species on the planet.

The way that humans adapt their behavior as they interact with surrounding environments is a crucial learning paradigm, which can be demonstrated by how a person drives a car.

When learning to drive, an inexperienced driver continuously performs steering, accelerating, and braking operations according to his perception of traffic environments.

Based on the built-in learning mechanisms in his brain, a driver improves his skills by choosing rewarded behaviors and avoiding punished behaviors.

After continuous practice for several months, the driver eventually becomes capable of reasonably considering safety, maneuverability, and ride comfort in his daily driving manipulation. This kind of trial-and- error process is an essential way for humans to learn new skills and develop intelligence.

For humans, how to learn a new skill is one of the most fundamental questions. This is also a fundamental question in artificial intelligence.

Reinforcement learning (RL) is a biologically inspired learning approach that focuses on finding optimal decision or control strategies for dynamic environments.

While traditional RL method focuses on learning from interactions with an unknown environment, modern RL method is more like a full- space optimizer for optimal control problem, which maximizes or minimizes a certain criterion to search for an optimal policy.

The power of RL mainly lies in its strong ability to solve complex and large-scale sequential decision problems.

Due to its potential in developing superhuman intelligent strategies, RL has attracted wide attention in a variety of interdisciplinary fields, including autonomous driving, game AI, robot control, and quantitative trading.

In 2017, RL was listed by the *MIT Technology Review* as one of the 10 breakthrough technologies.

History of Reinforcement learning

The history of RL dates back to the early days of cybernetics. Its development is mainly influenced by two separated fields:

(1) dynamic programming and (2) trial-and-error learning.

Dynamic programming (DP) was first introduced by Richard Bellman in the 1950s. DP is one of the three pillars of optimal control, along with the calculus of variations and Pontryagin's maximum principle.

The basis of DP is to divide a multistage optimization problem into a sequence of single-stage problems. As an abstraction of human and animal learning patterns, trial-and-error learning takes particular inspiration from Pavlovian conditioning and instrumental conditioning, which are rooted in the fields of psychology and neuroscience.

Some historical milestones of RL development are listed as follows:

- **1977, P. Werbos: DP with function approximation.**
- **1988, R. Sutton: Temporal difference learning**
- **1992, C. Watkins: Q-learning**
- **1994, G. Rummery, M. Niranjan: SARSA**
- **1996: D. Bertsekas & J. Tsitsiklis: RL convergence and stability**
- **1999, R. Sutton, D. McAllester, S. Singh: Policy gradient**
- **2014, D. Silver, G. Lever, N. Heess, et al.: Deterministic policy**
- **2015, V. Mnih, K. Kavukcuoglu, D. Silver, et al.: Deep Q-Network**
- **2015 to now, successes of Atari AI, AlphaGo, MuZero, and Racing Car AI**

The terminology in RL comes from a combination of notations in several fields, including optimal control, convex optimization, and statistical learning.

Reinforcement learning

Reinforcement learning, like many topics whose names end with "ing," such as machine learning and mountaineering, is simultaneously a problem, a class of solution methods that work well on the problem, and the field that studies this problem and its solution methods.

It is convenient to use a single name for all three things, but at the same time essential to keep the three conceptually separate.



In particular, the distinction between problems and solution methods is very important in reinforcement learning; failing to make this distinction is the source of many confusions.

We formalize the problem of reinforcement learning using ideas from dynamical systems theory, specifically, as the optimal control of incompletely-known Markov decision processes.

The basic idea is simply to capture the most important aspects of the real problem facing a learning agent interacting over time with its environment to achieve a goal, fig. 2.

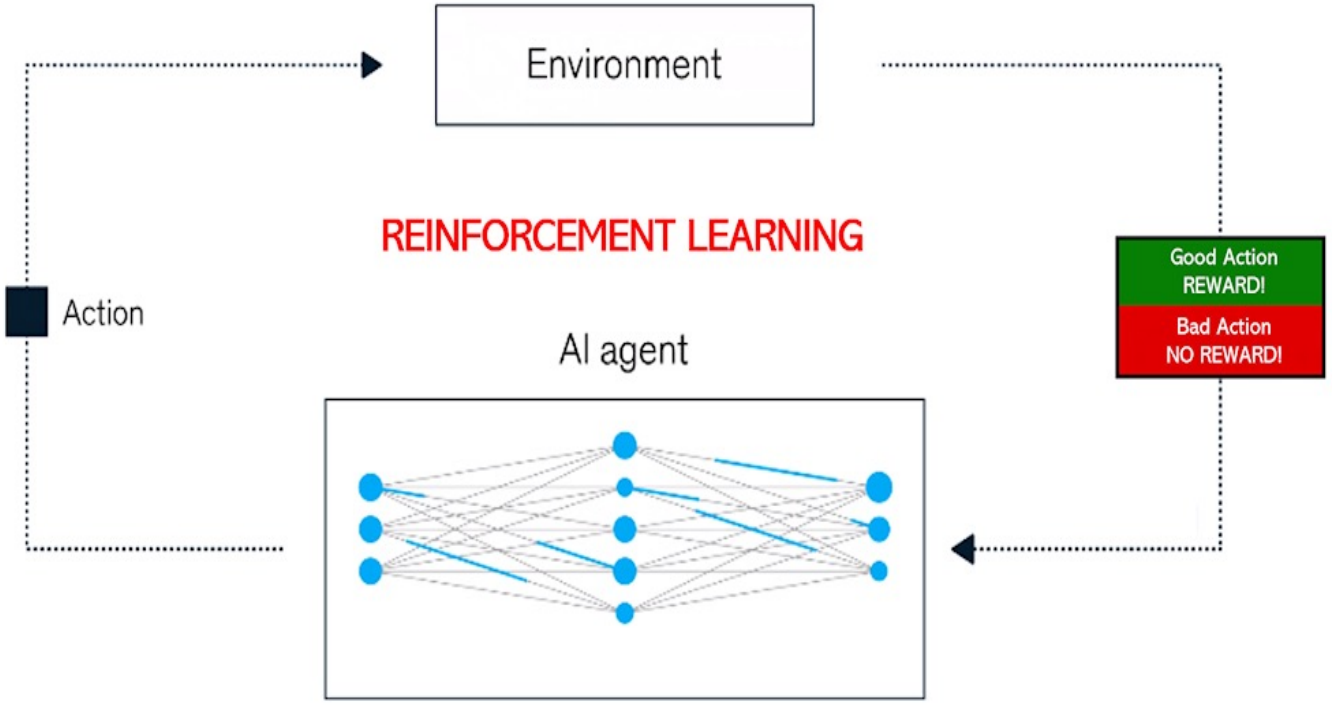


Fig. 2: Reinforcement learning

A learning agent must be able to sense the state of its environment to some extent and must be able to take actions that affect the state.

The agent also must have a goal or goals relating to the state of the environment.

Markov decision processes are intended to include just these three aspects—sensation, action, and goal—in their simplest possible forms without trivializing any of them.

Markov decision processes MDPs involves evaluative feedback, as in bandits, but also an associative aspect—choosing different actions in different situations.

MDPs are a classical formalization of sequential decision making, where actions influence not just immediate rewards, but also subsequent situations, or states, and through those future rewards.

Any method that is well suited to solving such problems we consider to be a reinforcement learning method.

Reinforcement learning is different from supervised learning, the kind of learning studied in most current research in the field of machine learning as in Fig 2.

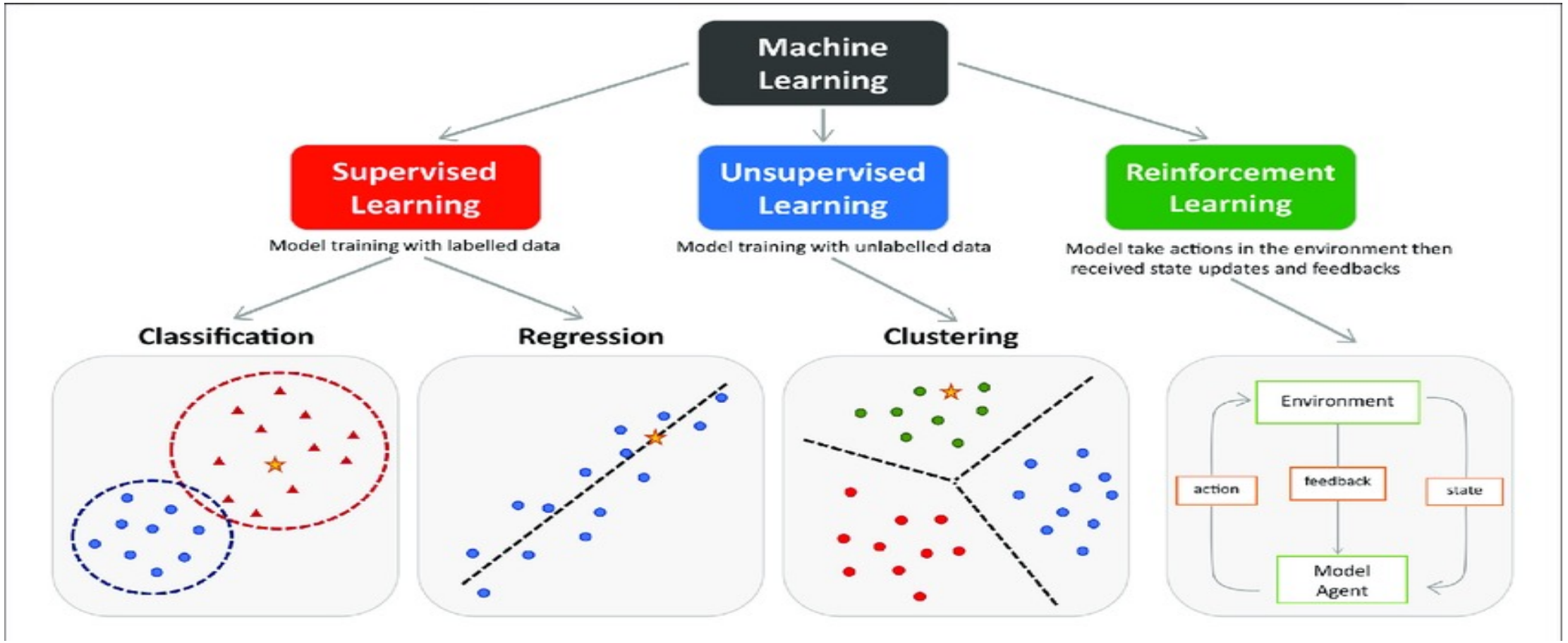


Fig. 2: Types of Machine Learning

Supervised learning is learning from a training set of labeled examples provided by a knowledgable external supervisor, fig 3.

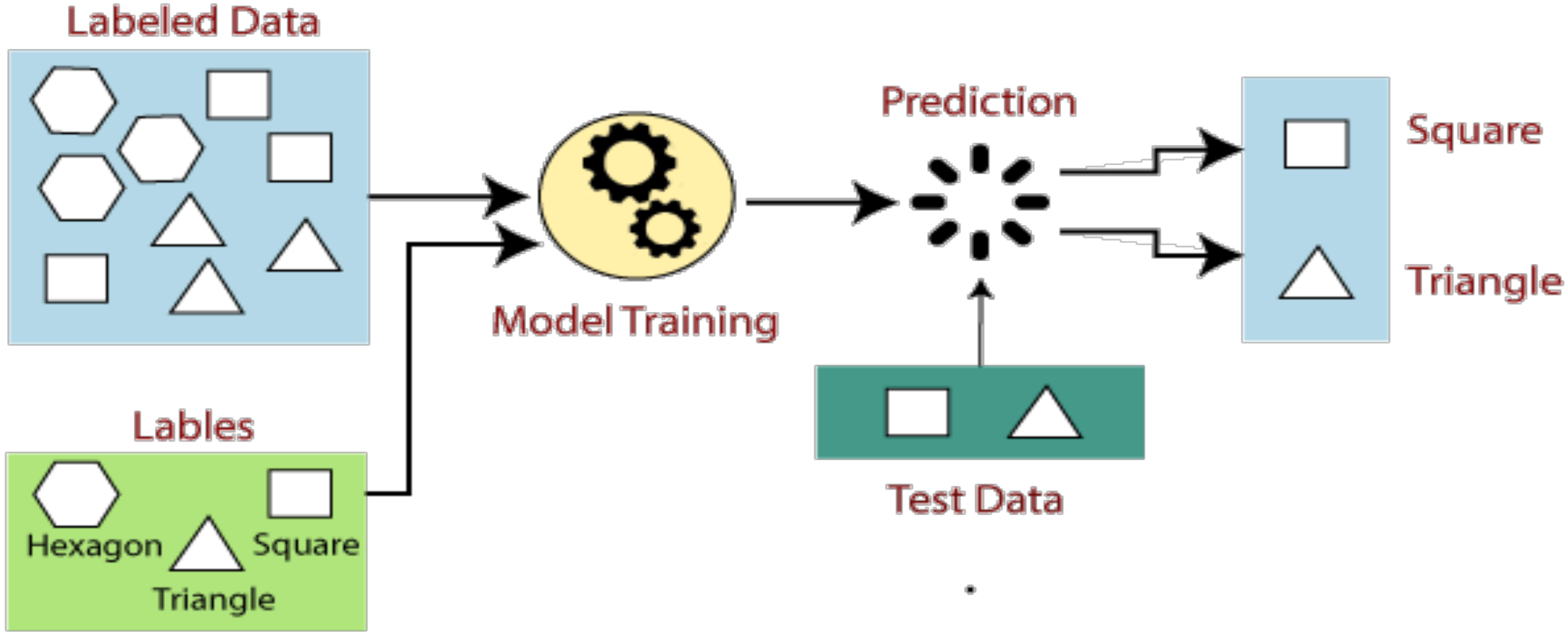


Fig. 3:Supervised Learning

Each example is a description of a situation together with a specification—the label—of the correct action the system should take to that situation, which is often to identify a category to which the situation belongs.

The object of this kind of learning is for the system to extrapolate, or generalize, its responses so that it acts correctly in situations not present in the training set.

This is an important kind of learning, but alone it is not adequate for learning from interaction.

In interactive problems it is often impractical to obtain examples of desired behavior that are both correct and representative of all the situations in which the agent has to act.

In uncharted territory—where one would expect learning to be most beneficial—an agent must be able to learn from its own experience

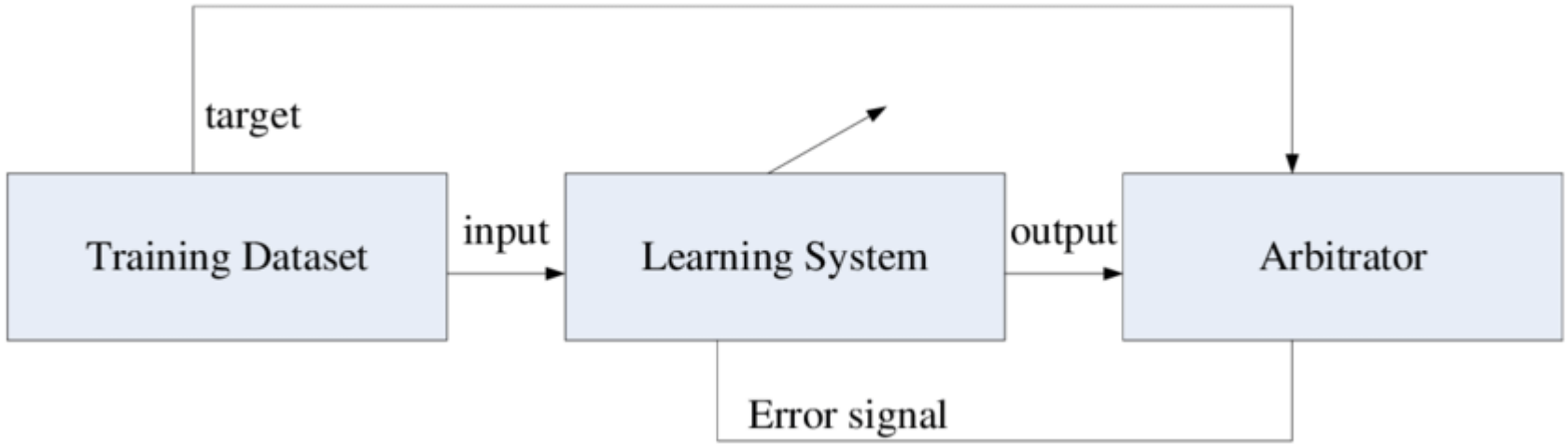


Fig. 4: Block diagram for supervised learning.

Reinforcement learning is also different from what machine learning researchers call unsupervised learning, fig. 5, which is typically about finding structure hidden in collections of unlabeled data.

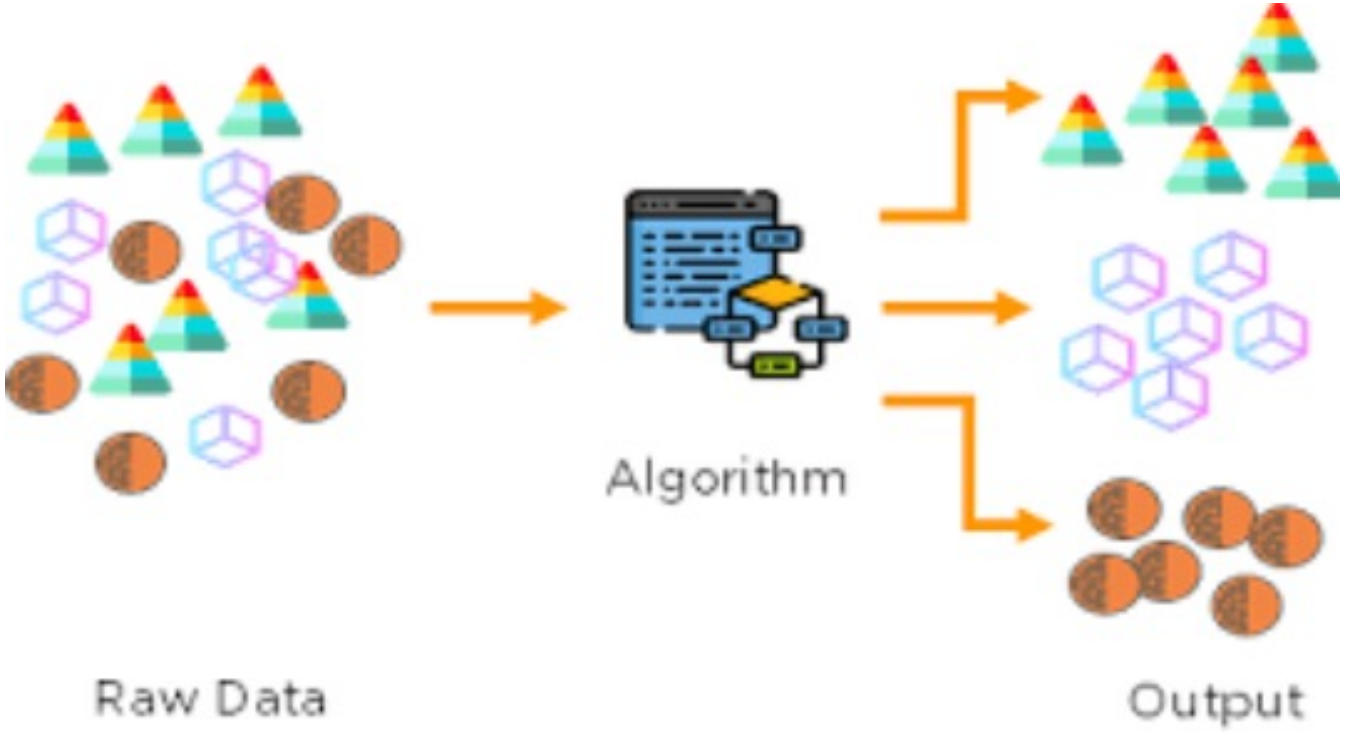


Fig. 5: Unsupervised Learning

The terms supervised learning and unsupervised learning, fig. 6, would seem to exhaustively classify machine learning paradigms, but they do not.

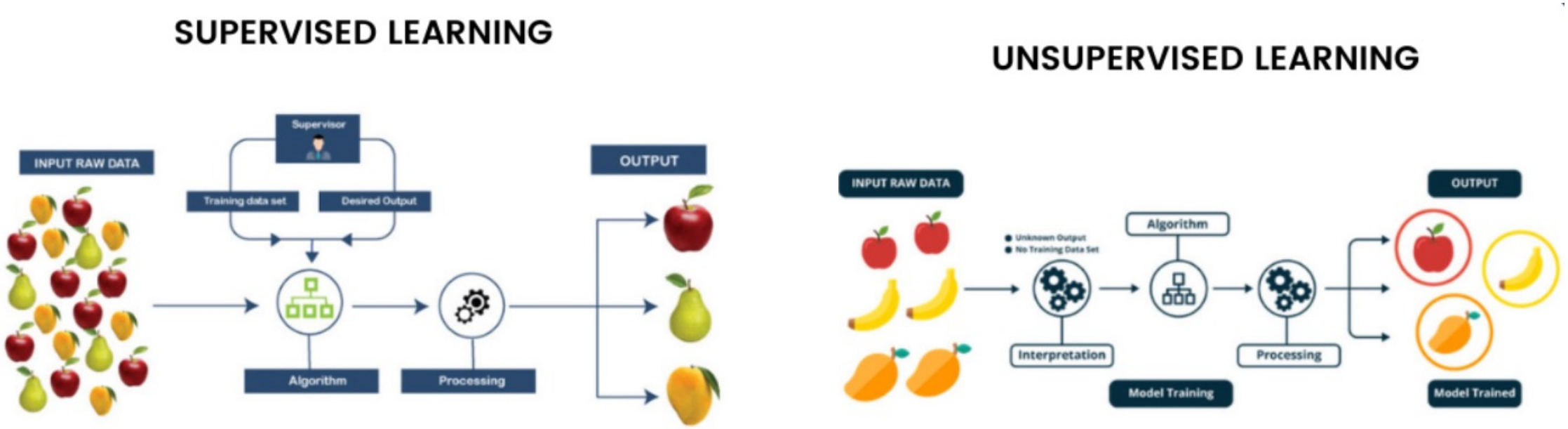
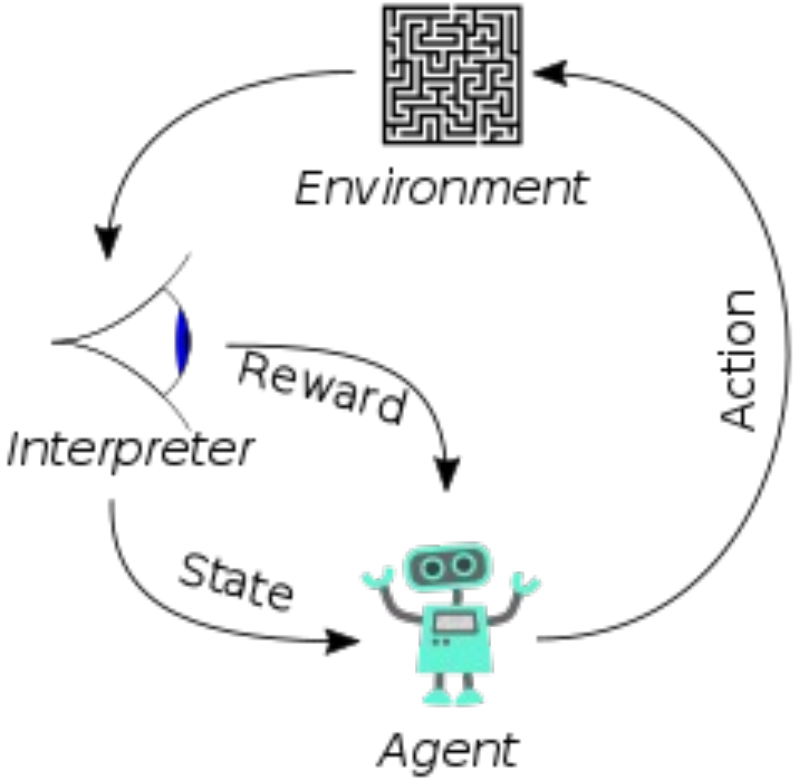


Fig. 6: Supervised and Unsupervised Learning

Although one might be tempted to think of reinforcement learning as a kind of unsupervised learning because it does not rely on examples of correct behavior, reinforcement learning is trying to maximize a reward signal instead of trying to find hidden structure.



Uncovering structure in an agent's experience can certainly be useful in reinforcement learning, but by itself does not address the reinforcement learning problem of maximizing a reward signal.

We therefore consider reinforcement learning to be a third machine learning paradigm, alongside supervised learning and unsupervised learning and perhaps other paradigms.



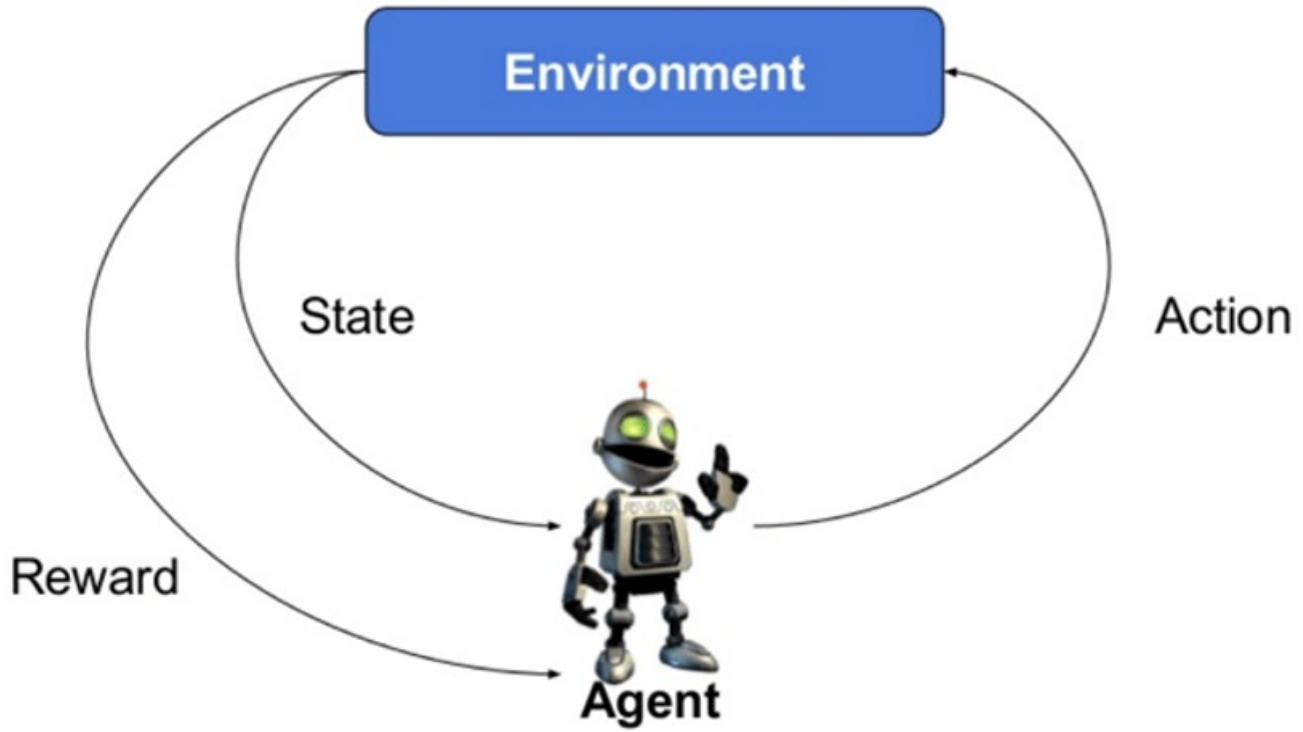
One of the challenges that arise in reinforcement learning, and not in other kinds of learning, is the trade-off between exploration and exploitation.

To obtain a lot of reward, a reinforcement learning agent must prefer actions that it has tried in the past and found to be effective in producing reward. But to discover such actions, it has to try actions that it has not selected before.

The agent has to exploit what it has already experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.

The dilemma is that neither exploration nor exploitation can be pursued exclusively without failing at the task.

The agent must try a variety of actions and progressively favor those that appear to be best.



Another key feature of reinforcement learning is that it explicitly considers the whole problem of a goal-directed agent interacting with an uncertain environment.

This is in contrast to many approaches that consider subproblems without addressing how they might fit into a larger picture.

For example, we have mentioned that much of machine learning research is concerned with supervised learning without explicitly specifying how such an ability would finally be useful.

Reinforcement learning takes the opposite tack, starting with a complete, interactive, goal-seeking agent.

All reinforcement learning agents have explicit goals, can sense aspects of their environments, and can choose actions to influence their environments.

Moreover, it is usually assumed from the beginning that the agent has to operate despite significant uncertainty about the environment it faces.

When reinforcement learning involves planning, it has to address the interplay between planning and real-time action selection, as well as the question of how environment models are acquired and improved.

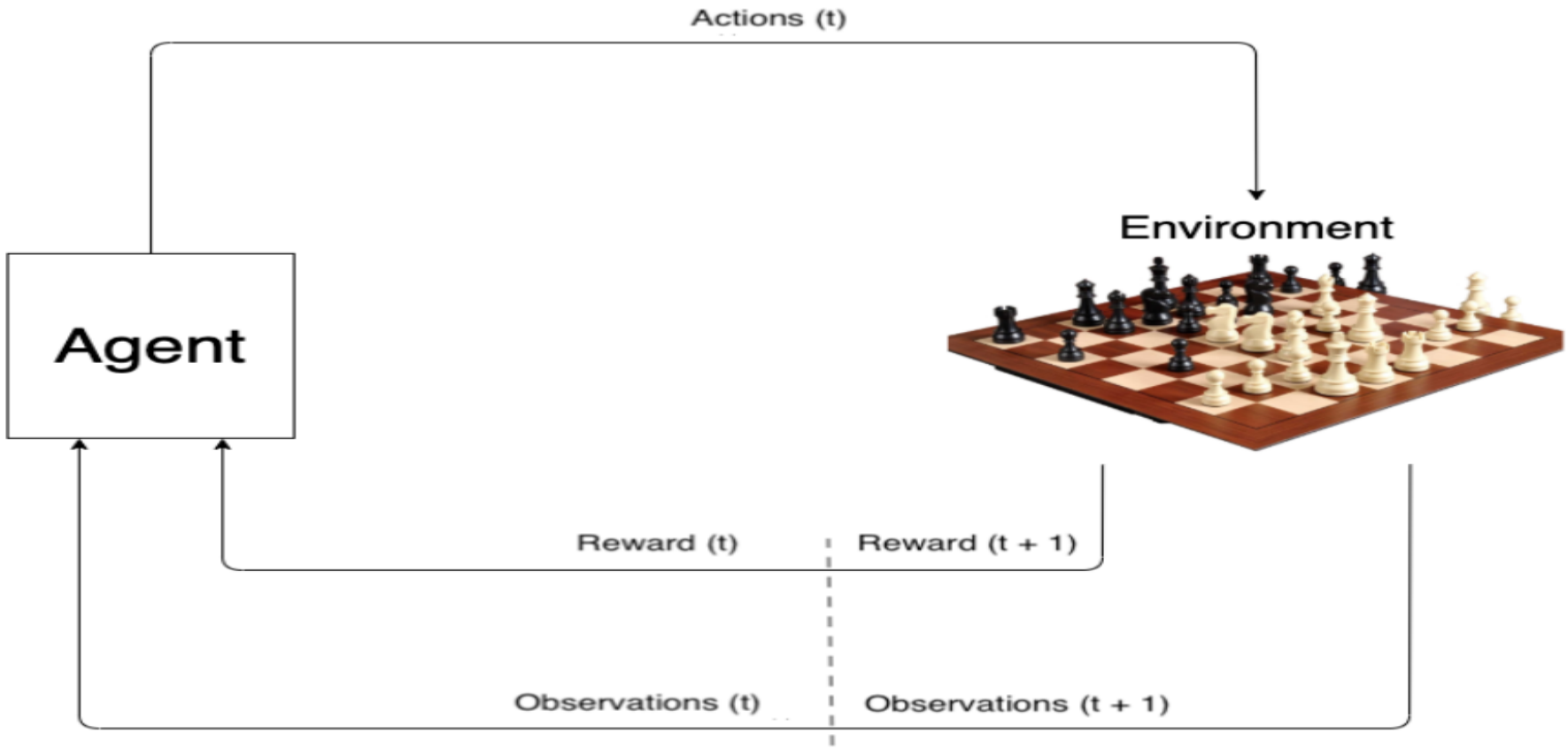
When reinforcement learning involves supervised learning, it does so for specific reasons that determine which capabilities are critical and which are not.

Examples

A good way to understand reinforcement learning is to consider some of the examples and possible applications that have guided its development.

- **A master chess player makes a move.**

The choice is informed both by planning — anticipating possible replies and counterreplies—and by immediate, intuitive judgments of the desirability of particular positions and moves.



- **An adaptive controller adjusts parameters of a petroleum refinery's operation in real time.**

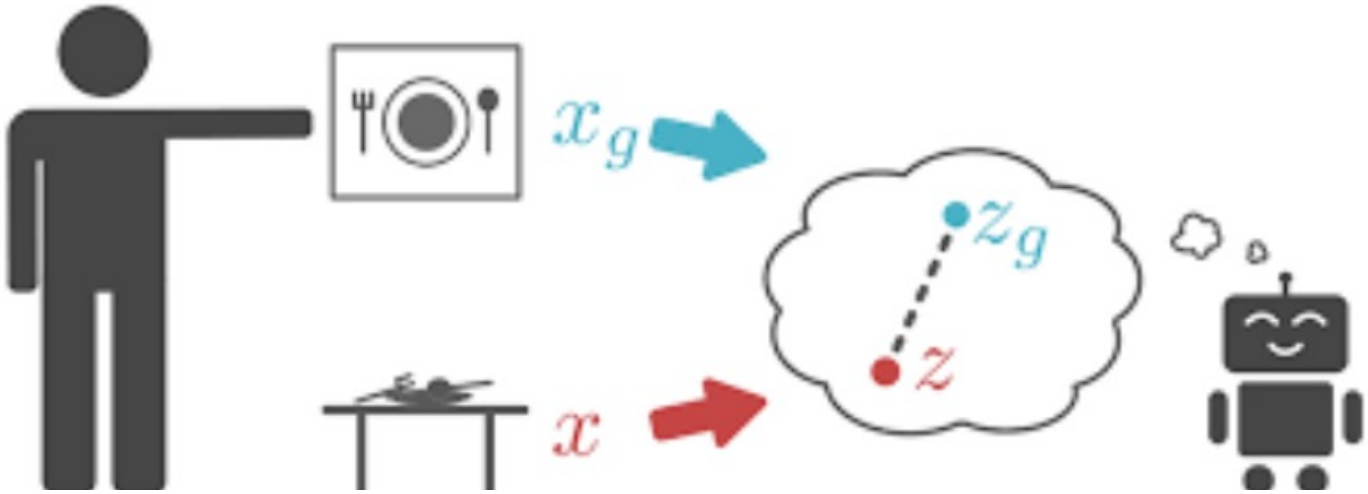
The controller optimizes the yield/cost/quality trade-off on the basis of specified marginal costs without sticking strictly to the set points originally suggested by engineers.

- **A gazelle calf struggles to its feet minutes after being born. Half an hour later it is running at 20 miles per hour.**

- **A mobile robot decides whether it should enter a new room in search of more trash to collect or start trying to find its way back to its battery recharging station. It makes its decision based on the current charge level of its battery and how quickly and easily it has been able to find the recharger in the past.**

- **Phil prepares his breakfast.**

Closely examined, even this apparently mundane activity reveals a complex web of conditional behavior and interlocking goal–subgoal relationships: walking to the cupboard, opening it, selecting a cereal box, then reaching for, grasping, and retrieving the box.



Other complex, tuned, interactive sequences of behavior are required to obtain a bowl, spoon, and milk carton.

Each step involves a series of eye movements to obtain information and to guide reaching and locomotion.

Rapid judgments are continually made about how to carry the objects or whether it is better to ferry some of them to the dining table before obtaining others.

Each step is guided by goals, such as grasping a spoon or getting to the refrigerator, and is in service of other goals, such as having the spoon to eat with once the cereal is prepared and ultimately obtaining nourishment.

Whether he is aware of it or not, Phil is accessing information about the state of his body that determines his nutritional needs, level of hunger, and food preferences.

These examples share features that are so basic that they are easy to overlook.

All involve interaction between an active decision-making agent and its environment, within which the agent seeks to achieve a goal despite uncertainty about its environment.

The agent's actions are permitted to affect the future state of the environment (e.g., the next chess position, the level of reservoirs of the refinery, the robot's next location and the future charge level of its battery), thereby affecting the actions and opportunities available to the agent at later times.

At the same time, in all of these examples the effects of actions cannot be fully predicted; thus the agent must monitor its environment frequently and react appropriately.

For example, Phil must watch the milk he pours into his cereal bowl to keep it from overflowing.

All these examples involve goals that are explicit in the sense that the agent can judge progress toward its goal based on what it can sense directly.

The chess player knows whether or not he wins, the refinery controller knows how much petroleum is being produced, the gazelle calf knows when it falls, the mobile robot knows when its batteries run down, and Phil knows whether or not he is enjoying his breakfast.

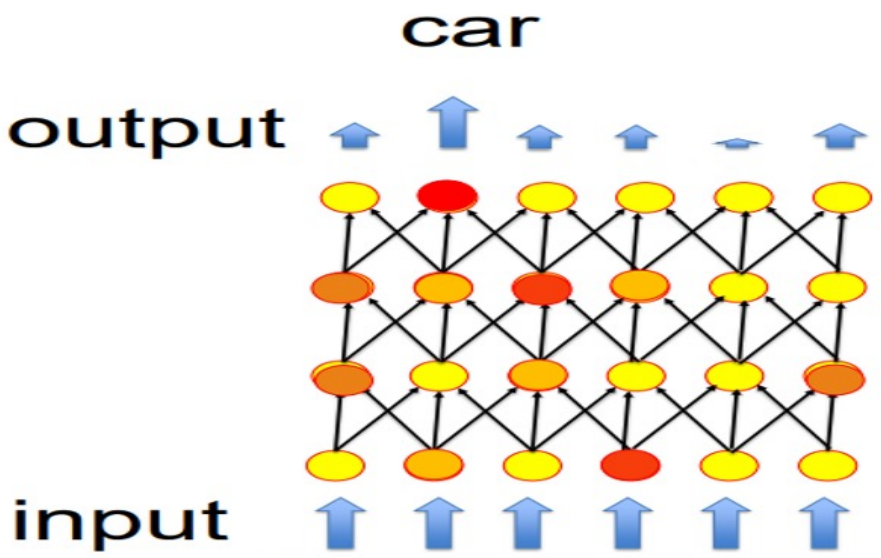
In all of these examples the agent can use its experience to improve its performance over time.

The chess player refines the intuition he uses to evaluate positions, thereby improving his play; the gazelle calf improves the efficiency with which it can run; Phil learns to streamline making his breakfast.

The knowledge the agent brings to the task at the start—either from previous experience with related tasks or built into it by design or evolution—influences what is useful or easy to learn, but interaction with the environment is essential for adjusting behavior to exploit specific features of the task.

Artificial Neural Networks for classification

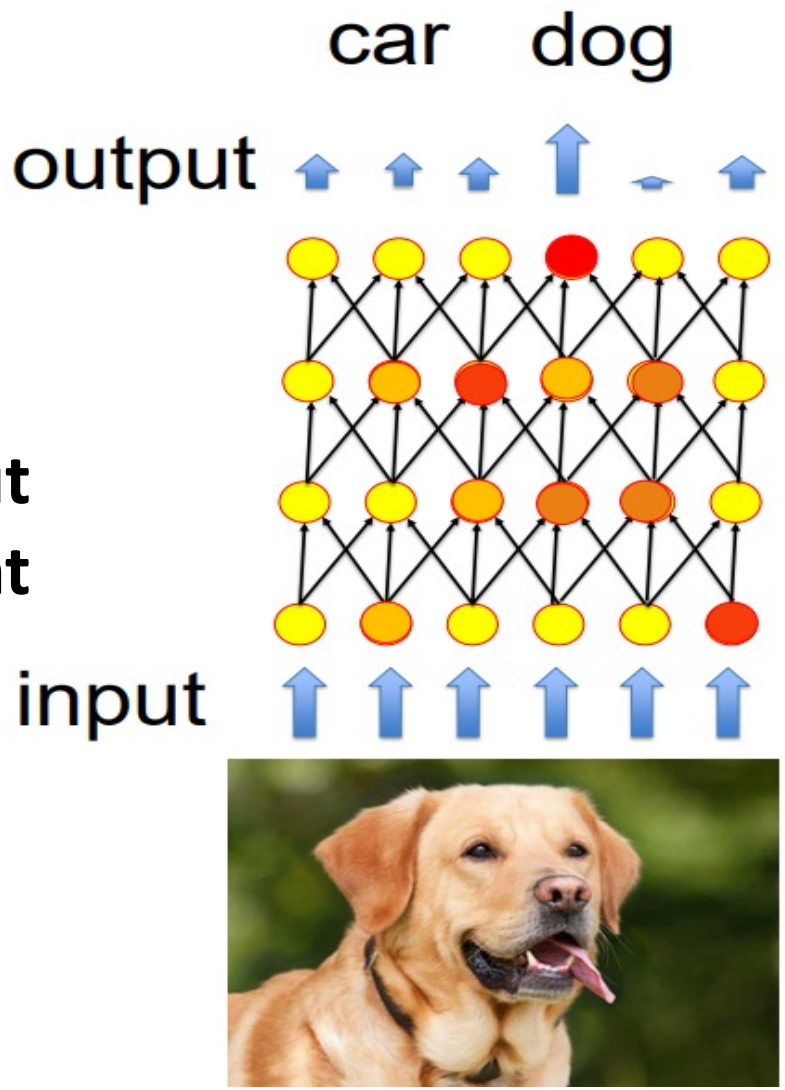
At the bottom of the network is an input.
It goes across several neuronal layers.
A feedforward network is an architecture where all connections are made from the bottom to the uppermost layer.
The result is a collection of neurons that represent several "classes."
If the input picture contains a car, the ideal network should react by activating the neuron that stands for "car".





Learning objective:

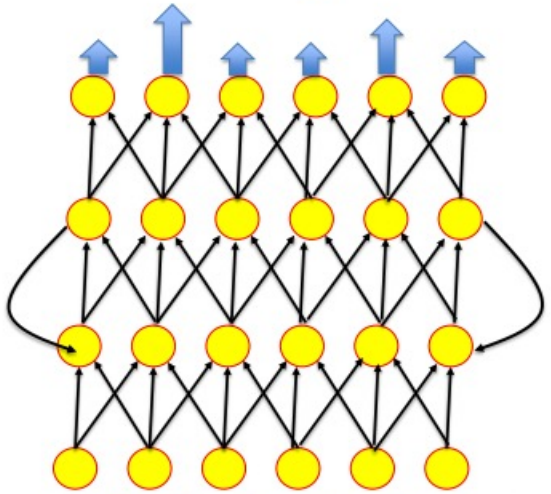
Modify connections so that the output class is accurate (for each subsequent input).



- **The purpose of learning is to modify the connection weights so that the output class is accurate for each subsequent input.**
- **The 'dog'-neuron should react if the input is a dog.**
- **The 'vehicle'-neuron should react if the input is a car.**

Deep networks with recurrent connections

‘a man sitting on a couch with a dog’





Network desribes the image with the words:

‘a man sitting on a couch with a dog’

An amazing example of sequence production with a recurrent neural network is this network which looks at a static image and outputs the spoken sentence: ‘A man is sitting on a couch with a dog’.

A sentence is a temporal sequence of words – and importantly, a sequence that follows grammatical rules.

Sequence learning requires recurrent connections (feedback connections), in contrast to the feedforward architecture that we have seen so far.

And yes, recurrent neural networks can implicitly pick up the statistical rules for sentence formation if they are trained on a sufficiently large data set containing millions of examples.

Artificial Neural Networks for action learning

Learning from mistakes, And from successes

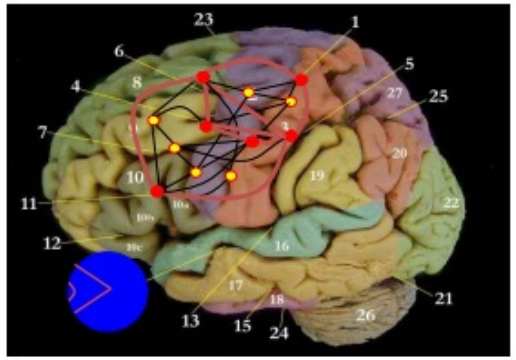




Missing: Value of action

- ‘goodie’ for dog

- ‘success’ - ‘compliment’



Reinforcement learning = learning based on reward



We learn actions by trial and error exploiting rather general feedback: reward or praise on one side, pleasure and pain on the other side. Important is the notion of value of an action.

Learning actions or sequences of actions based on ‘reward’ is very different from classification: it falls in the field of ‘Reinforcement Learning’.

Deep reinforcement learning

Artificial neural network (AlphaZero)

discovers different strategies by

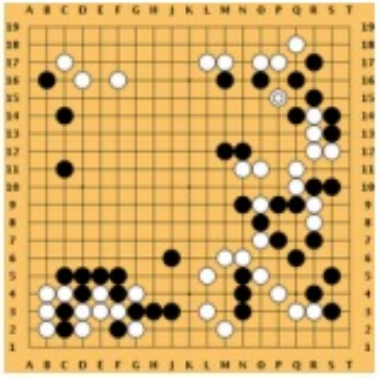
playing against itself. In Go, it beats Lee Sedol



Chess



Go



The same kind of ideas (learning by reward) have also been implemented in artificial neural networks that are trained by reinforcement learning.

In a game such a Chess or Go, the reward signal is only given once at the very end of the game: positive reward if the game is won, and negative reward if it is lost.

This very sparse reward information is sufficient to train an artificial neural network to a level where it can win against grand masters in chess or Go.

To improve performance, each network plays against a copy of itself. By doing so it discovers good strategies (such as openings in chess)

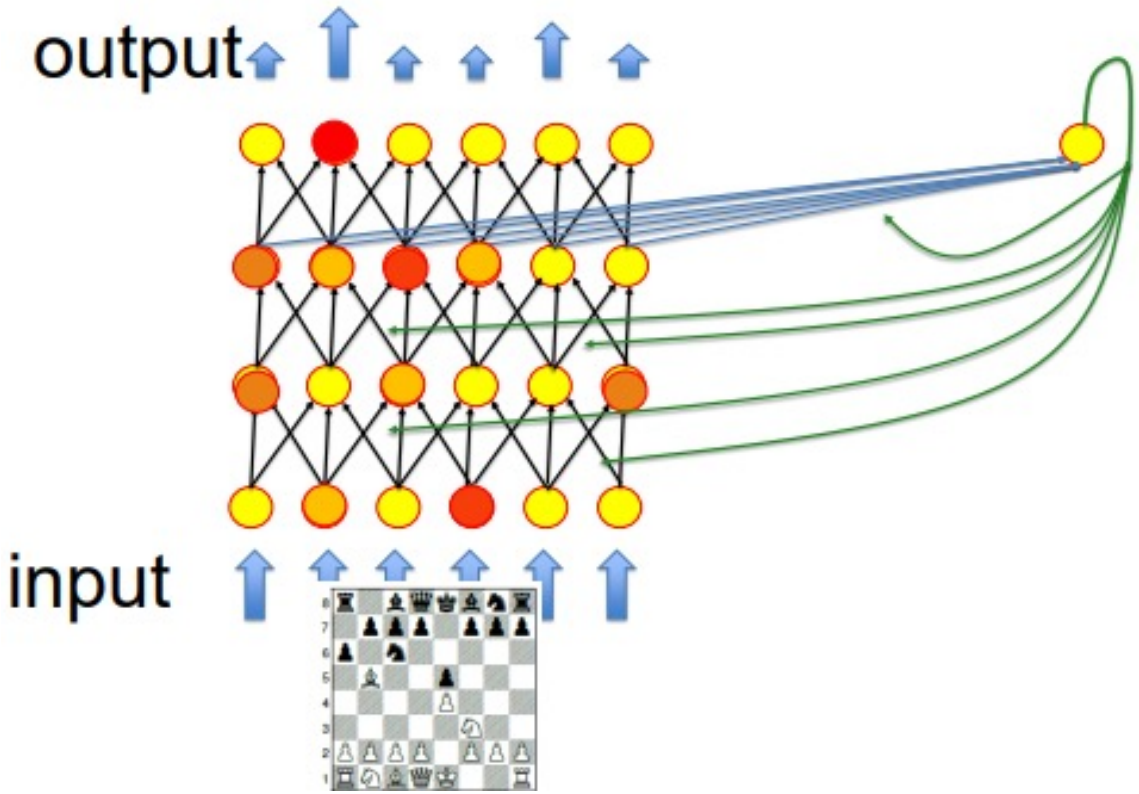
Network for choosing action

action:

Advance king

2nd output for **value** of action:

probability to win



learning:

- change connections

aim:

- Predict value of position
- Choose next action to win

Schematically, the artificial neural network takes the position of chess as input.

There are two types of outputs:

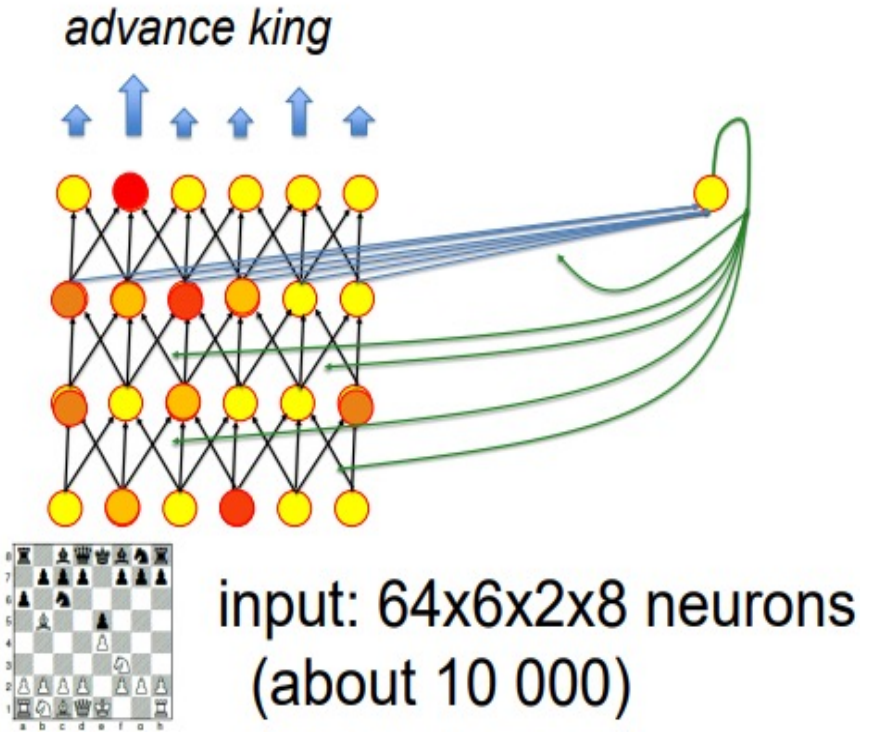
- **The main outputs are the actions such as ‘move king to the right’**
- **An auxiliary output predicts the ‘value’ of each state.**

It can be used to explore possible next positions so as to pick the one with the highest value.

- The value can be interpreted as the probability to win (given the position) In the theory of reinforcement learning, positions are also called ‘states’

Training 44Mio games (9 hours) Planning: potential sequences (during 1s before playing next action)

output: 4672 actions



Since there are many different positions, the number of input neurons is in the range of ten thousand:

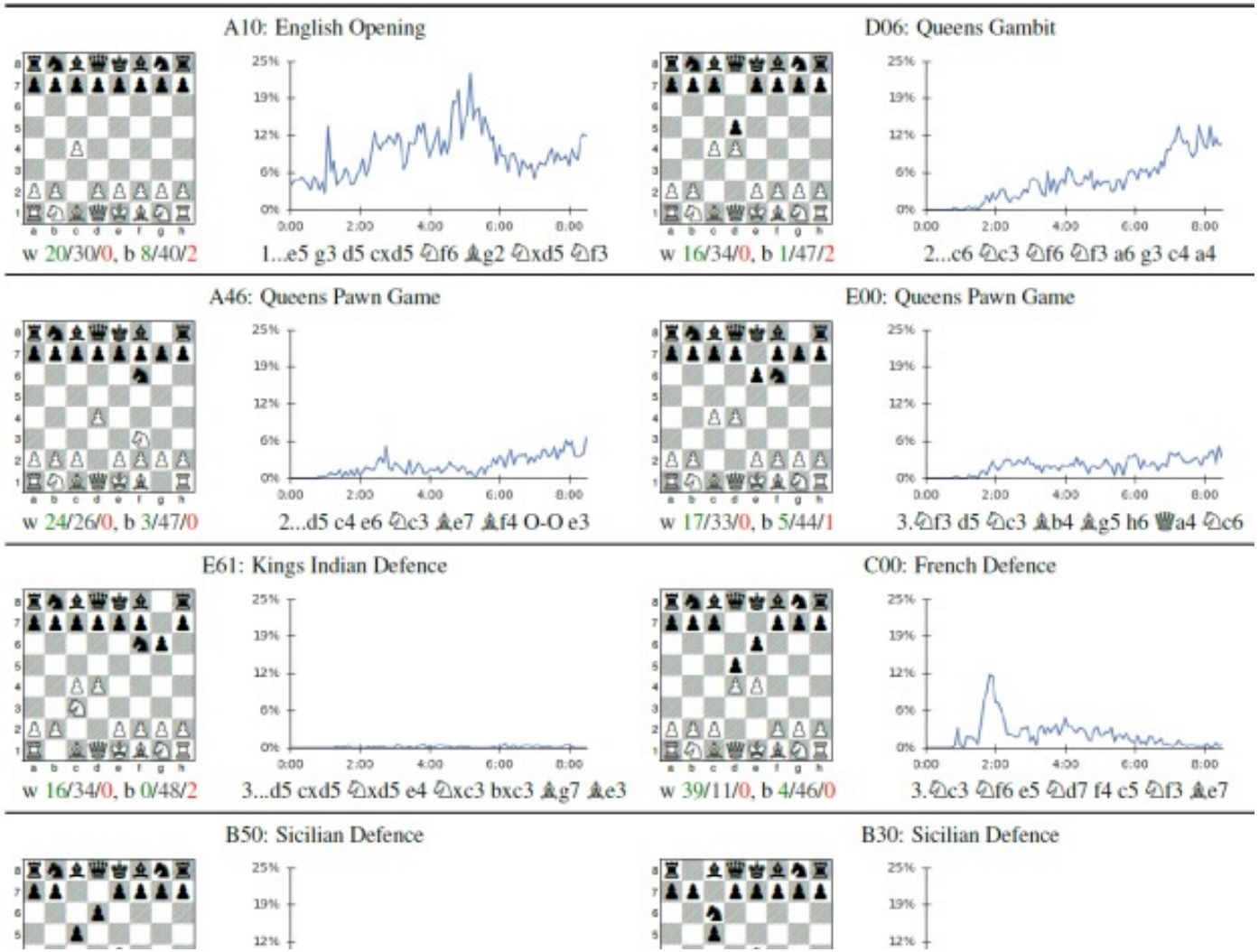
On each of 64 positions there can be one of 6 different ‘figures’ (king, knight, bishop) of 2 different colors.

To avoid repetitions the 8 last time steps are used as input.

Training is done by playing against itself in 44 million games.

The allotted computer time for planning the next action is 1s.

Deep reinforcement learning (alpha zero)



Chess:

-discovers classic openings

-beats best human players

-beats best classic AI algorithms

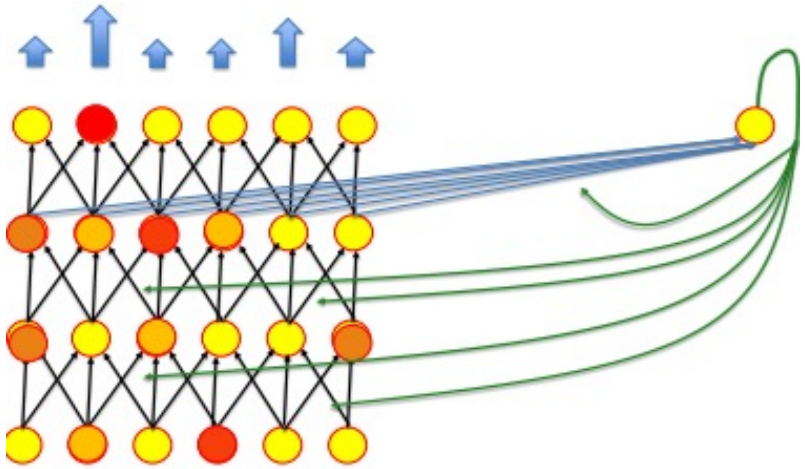
After training for 44 Milllion self-play games, the algorithm matches or beats classical AI algorithms for chess.

Interestingly, it ‘discovers’ well-known strategies for openings, corresponding closely to well known openings in textbooks on chess. When trained on Go it beats the world champions.

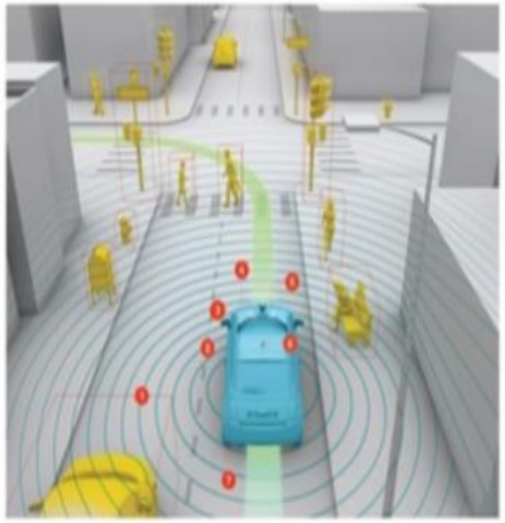
Similar reinforcement learning algorithms are also used to train self driving cars.

Self-driving cars

advance and accerate







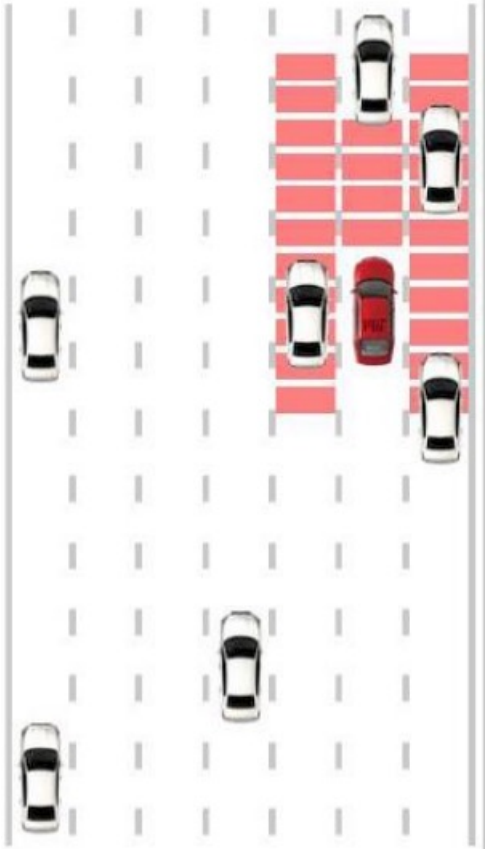


External

1. Radar
2. Visible-light camera
3. LIDAR
4. Infrared camera
5. Stereo vision
6. GPS/IMU
7. CAN
8. Audio

Internal

1. Visible-light camera
2. Infrared camera
3. Audio



Road Overlay:

Safety System

Value: security, duration of travel

H.W

- **What's the idea of learning in reinforcement learning?**
- **What is the role of agent in reinforcement learning?**
- **How can we distinguish between the learning processes Reinforcement learning, supervised learning, and unsupervised learning?**
- **Give a clear example that shows the role of the agent in the reinforcement learning process.**
- **Describe the mechanism of Artificial Neural Networks for classification.**

References

- **Sutton, Richard S., and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 2018.**
- **Dong, H., Dong, H., Ding, Z., Zhang, S., & Chang, T. (2020). Deep Reinforcement Learning. Singapore: Springer Singapore.**
- **Bishop, Christopher M. Neural networks for pattern recognition. Oxford university press, 1995.**
- **Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. Deep learning. MIT press, 2016.**
- **Sutton, Richard S., and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 20**
- **Sutton, Richard S., and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 2018.**